

APOSTILA DE ENSINO

SQL Server

Do Básico ao Avançado

Dataset baseado na arquitetura de dados da Uber

corridas motoristas usuarios pagamentos avaliacoes

11 Módulos | Conteúdo Técnico | Documentação Oficial Microsoft

Sumario

Modulo 1 — Fundamentos de Bancos Relacionais

- Modelo relacional
- SQL vs NoSQL
- Tipos de dados do SQL Server

Modulo 2 — Instalacao e Configuracao

- SQL Server via Docker
- SSMS e Azure Data Studio
- Criacao do banco uber_db

Modulo 3 — DDL — Criacao da Estrutura de Dados

- CREATE TABLE com constraints
- Schema completo Uber (8 tabelas)
- IDENTITY, UNIQUE, CHECK, FK

Modulo 4 — DML — Operacoes CRUD

- INSERT / SELECT / UPDATE / DELETE
- Paginacao com OFFSET FETCH
- Operadores de filtragem

Modulo 5 — Consultas Avancadas

- Tipos de JOIN e quando usar
- Subqueries
- CASE WHEN e funcoes nativas

Modulo 6 — Indices e Performance

- Clustered vs Non-Clustered
- Indices compostos e filtrados
- Execution Plan e STATISTICS IO

Modulo 7 — Stored Procedures, Funcoes e Transactions

- SP com TRY/CATCH e THROW
- Funcoes escalares e TVF
- BEGIN TRANSACTION / ROLLBACK

Modulo 8 — Aggregations e Window Functions

- GROUP BY, HAVING, ROLLUP
- ROW_NUMBER, RANK, DENSE_RANK
- LAG, LEAD, SUM OVER PARTITION

Modulo 9 — Views, CTEs e Queries Avancadas

- CREATE VIEW
- WITH CTE e CTEs recursivas
- MERGE e PIVOT

Modulo 10 — Exportacao, Backup e Automacao

- BCP e BULK INSERT
- BACKUP DATABASE + Restore

- Estrategia Full/Diferencial/Log

Modulo 11 — SQL Server com Docker

- docker-compose.yml completo
- Volumes e persistencia
- Inicializacao automatica do schema

Fundamentos de Bancos Relacionais

Bancos de dados relacionais organizam informacoes em tabelas com linhas e colunas, garantindo integridade por meio de chaves primarias, chaves estrangeiras e restricoes. O SQL Server, da Microsoft, e um dos SGBDs mais adotados em ambientes corporativos, oferecendo alta disponibilidade, seguranca robusta e integracao nativa com o ecossistema Azure.

SQL vs NoSQL — Quando usar cada um?

Característica	SQL (SQL Server)	NoSQL (MongoDB)
Estrutura	Tabelas, linhas e colunas com schema rigido	Colecoes e documentos com schema flexivel
Relacionamentos	JOINS com integridade referencial	Documentos aninhados ou referencias manuais
Transacoes	ACID nativo e robusto	Suporte limitado / eventual consistency
Escalabilidade	Vertical (scale-up)	Horizontal (scale-out)
Consultas	SQL declarativo e padronizado	Query language proprietaria
Caso Uber	Transacoes financeiras, auditoria, relatorios	Logs de geolocalizacao em tempo real

Principais tipos de dados no SQL Server

Tipo	Descricao	Exemplo de uso no dataset Uber
INT / BIGINT	Inteiros de 32 e 64 bits	corrida_id, usuario_id
DECIMAL(p,s)	Numero exato com precisao e escala definidas	valor_final DECIMAL(10,2)
VARCHAR(n)	Texto variavel ASCII, mais leve	email, status, metodo_pagamento
NVARCHAR(n)	Texto variavel Unicode (UTF-16) para acentos	nome, origem_endereco
DATETIME2	Data e hora com precisao ate 100ns	data_solicitacao, data_fim
BIT	Booleano (0 ou 1)	motorista_ativo, cidade_ativa
TINYINT	Inteiro de 0 a 255	nota em avaliacoes
CHAR(n)	Texto de tamanho fixo	estado CHAR(2), placa CHAR(8)

SMALLINT	Inteiro -32768 a 32767	ano do veiculo
UNIQUEIDENTIFIER	GUID de 128 bits	trip_uuid para sistemas distribuidos

Conceitos fundamentais de modelagem relacional

Normalizacao e o processo de organizar as colunas e tabelas para reduzir redundancia e melhorar a integridade. As tres primeiras formas normais (1FN, 2FN, 3FN) cobrem a maioria dos cenarios de aplicacoes transacionais. Em ambientes de BI e DW, a desnormalizacao controlada (star schema, snowflake) e preferida para performance de leitura.

Forma Normal	Regra	Violacao classica
1FN	Sem grupos repetidos, cada campo atômico	Coluna 'telefone1,telefone2' em uma string
2FN	Sem dependencia parcial da chave primaria	Guardar nome do produto em tabela de itens_pedido
3FN	Sem dependencia transitiva entre nao-chaves	Guardar cidade e estado do CEP na tabela de clientes

[Documentacao oficial — Tipos de dados \(T-SQL\)](#)

[Documentacao oficial — Comparacao SQL Server Editions](#)

[Documentacao oficial — Guia de normalizacao de banco de dados](#)

Instalacao e Configuracao

Instalacao via Docker (recomendado para desenvolvimento)

```
# Baixar imagem oficial do SQL Server 2022
docker pull mcr.microsoft.com/mssql/server:2022-latest

# Executar container com variaveis de ambiente
docker run -e 'ACCEPT_EULA=Y' \
-e 'SA_PASSWORD=UberWorkshop@2024' \
-p 1433:1433 --name sqlserver \
-d mcr.microsoft.com/mssql/server:2022-latest

# Verificar status do container
docker ps | grep sqlserver

# Conectar via sqlcmd dentro do container
docker exec -it sqlserver /opt/mssql-tools/bin/sqlcmd \
-S localhost -U SA -P 'UberWorkshop@2024'
```

Aspecto	Instalacao Convencional	Docker
Facilidade	Media — requer configuracao do SO	Alta — um comando sobe o ambiente
Isolamento	Compartilha recursos do host	Totalmente isolado em container
Portabilidade	Baixa — especifico do SO	Alta — roda igual em qualquer maquina
Reproducibilidade	Trabalhosa	Total — docker-compose up
Uso recomendado	Producao on-premise	Desenvolvimento e testes

Criacao do banco e verificacoes iniciais

```
-- Criar banco de dados com collation explicita
CREATE DATABASE uber_db
COLLATE Latin1_General_CI_AI;
-- CI = Case Insensitive | AI = Accent Insensitive
USE uber_db;

-- Verificar bancos existentes
SELECT name, create_date, collation_name, compatibility_level
FROM sys.databases
ORDER BY create_date DESC;

-- Verificar versao instalada
SELECT @@VERSION;

-- Verificar configuracoes do banco
SELECT name, value_in_use
FROM sys.configurations
WHERE name IN ('max degree of parallelism',
'cost threshold for parallelism',
'max server memory (MB)');
```

Dica: Para Windows, baixe o SQL Server Developer Edition gratuitamente em <https://www.microsoft.com/sql-server> — possui todos os recursos do Enterprise sem custo para uso em desenvolvimento e testes.

[Documentacao oficial — Instalacao SQL Server no Docker](#)

[Documentacao oficial — Azure Data Studio \(ferramenta gratuita\)](#)

[Documentacao oficial — CREATE DATABASE](#)

DDL — Criacao da Estrutura de Dados

O dataset simula parte da arquitetura de dados transacionais da Uber: usuarios (passageiros), motoristas, veiculos, corridas, pagamentos e avaliacoes. As tabelas seguem boas praticas de normalizacao e uso de constraints para garantir integridade referencial.

Schema completo — uber_db


```

USE uber_db;

-- 1. CIDADES
CREATE TABLE cidades (
cidade_id INT IDENTITY(1,1) PRIMARY KEY,
nome NVARCHAR(100) NOT NULL,
estado CHAR(2) NOT NULL,
pais CHAR(3) NOT NULL DEFAULT 'BRA',
fuso VARCHAR(40) NOT NULL DEFAULT 'America/Sao_Paulo',
ativa BIT NOT NULL DEFAULT 1
);

-- 2. USUARIOS (passageiros)
CREATE TABLE usuarios (
usuario_id INT IDENTITY(1,1) PRIMARY KEY,
nome NVARCHAR(120) NOT NULL,
email VARCHAR(180) NOT NULL,
telefone VARCHAR(20) NOT NULL,
data_cadastro DATETIME2 NOT NULL DEFAULT GETDATE(),
ativo BIT NOT NULL DEFAULT 1,
cidade_id INT NOT NULL,
CONSTRAINT uq_usuario_email UNIQUE (email),
CONSTRAINT fk_usuario_cidade FOREIGN KEY (cidade_id)
REFERENCES cidades(cidade_id)
);

-- 3. MOTORISTAS
CREATE TABLE motoristas (
motorista_id INT IDENTITY(1,1) PRIMARY KEY,
nome NVARCHAR(120) NOT NULL,
email VARCHAR(180) NOT NULL,
cnh VARCHAR(20) NOT NULL,
data_cadastro DATETIME2 NOT NULL DEFAULT GETDATE(),
avaliacao_media DECIMAL(3,2) NULL,
ativo BIT NOT NULL DEFAULT 1,
cidade_id INT NOT NULL,
CONSTRAINT uq_motorista_email UNIQUE (email),
CONSTRAINT uq_motorista_cnh UNIQUE (cnh),
CONSTRAINT fk_motorista_cidade FOREIGN KEY (cidade_id)
REFERENCES cidades(cidade_id)
);

```

```

-- 4. VEICULOS
CREATE TABLE veiculos (
veiculo_id INT IDENTITY(1,1) PRIMARY KEY,
motorista_id INT NOT NULL,
marca VARCHAR(50) NOT NULL,
modelo VARCHAR(80) NOT NULL,
ano SMALLINT NOT NULL,
placa CHAR(8) NOT NULL,
categoria VARCHAR(20) NOT NULL
CHECK (categoria IN ('UberX','UberBlack','UberComfort','Moto')),
ativo BIT NOT NULL DEFAULT 1,
CONSTRAINT uq_veiculo_placa UNIQUE (placa),
CONSTRAINT fk_veiculo_motorista FOREIGN KEY (motorista_id)
REFERENCES motoristas(motorista_id)
);

-- 5. CORRIDAS
CREATE TABLE corridas (
corrida_id INT IDENTITY(1,1) PRIMARY KEY,
usuario_id INT NOT NULL,
motorista_id INT NOT NULL,
veiculo_id INT NOT NULL,
cidade_id INT NOT NULL,
status VARCHAR(20) NOT NULL
CHECK (status IN ('solicitada','aceita','em_andamento',
'concluida','cancelada')),
origem_endereco NVARCHAR(200) NOT NULL,
destino_endereco NVARCHAR(200) NOT NULL,
data_solicitacao DATETIME2 NOT NULL DEFAULT GETDATE(),
data_inicio DATETIME2 NULL,
data_fim DATETIME2 NULL,
distancia_km DECIMAL(8,3) NULL,
duracao_minutos INT NULL,
valor_base DECIMAL(10,2) NULL,
valor_final DECIMAL(10,2) NULL,
surge_multiplier DECIMAL(4,2) NOT NULL DEFAULT 1.00,
CONSTRAINT fk_corrida_usuario FOREIGN KEY (usuario_id)
REFERENCES usuarios(usuario_id),
CONSTRAINT fk_corrida_motorista FOREIGN KEY (motorista_id)
REFERENCES motoristas(motorista_id),
CONSTRAINT fk_corrida_veiculo FOREIGN KEY (veiculo_id)
REFERENCES veiculos(veiculo_id),
CONSTRAINT fk_corrida_cidade FOREIGN KEY (cidade_id)
REFERENCES cidades(cidade_id)
);

```

```

-- 6. PAGAMENTOS
CREATE TABLE pagamentos (
  pagamento_id INT IDENTITY(1,1) PRIMARY KEY,
  corrida_id INT NOT NULL UNIQUE,
  metodo VARCHAR(30) NOT NULL
  CHECK (metodo IN ('credito','debito','pix','carteira_uber')),
  valor DECIMAL(10,2) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pendente'
  CHECK (status IN ('pendente','aprovado','recusado','estornado')),
  data_pagamento DATETIME2 NULL,
  CONSTRAINT fk_pagamento_corrida FOREIGN KEY (corrida_id)
  REFERENCES corridas(corrida_id)
);

-- 7. AVALIACOES
CREATE TABLE avaliacoes (
  avaliacao_id INT IDENTITY(1,1) PRIMARY KEY,
  corrida_id INT NOT NULL,
  avaliador_tipo VARCHAR(10) NOT NULL
  CHECK (avaliador_tipo IN ('usuario','motorista')),
  nota TINYINT NOT NULL CHECK (nota BETWEEN 1 AND 5),
  comentario NVARCHAR(500) NULL,
  data_avaliacao DATETIME2 NOT NULL DEFAULT GETDATE(),
  CONSTRAINT fk_avaliacao_corrida FOREIGN KEY (corrida_id)
  REFERENCES corridas(corrida_id)
);

-- 8. PROMOCOES
CREATE TABLE promocoas (
  promocao_id INT IDENTITY(1,1) PRIMARY KEY,
  codigo VARCHAR(20) NOT NULL UNIQUE,
  descricao NVARCHAR(200) NOT NULL,
  desconto_pct DECIMAL(5,2) NOT NULL
  CHECK (desconto_pct BETWEEN 1 AND 100),
  validade DATE NOT NULL,
  uso_maximo INT NOT NULL DEFAULT 1,
  usos_realizados INT NOT NULL DEFAULT 0,
  ativa BIT NOT NULL DEFAULT 1
);

```

Constraints — Referencia rapida

Constraint	Funcao	Exemplo no schema
PRIMARY KEY	Identifica unicamente cada linha	corrida_id INT IDENTITY PRIMARY KEY

FOREIGN KEY	Garante que o valor existe na tabela pai	REFERENCES cidades(cidade_id)
UNIQUE	Proibe duplicatas em uma coluna	CONSTRAINT uq_usuario_email UNIQUE (email)
CHECK	Valida valor contra uma expressao	CHECK (nota BETWEEN 1 AND 5)
DEFAULT	Valor automatico quando nao informado	data_cadastro DATETIME2 DEFAULT GETDATE()
NOT NULL	Proibe valores nulos	nome NVARCHAR(120) NOT NULL
IDENTITY	Auto-incremento gerenciado pelo SQL Server	INT IDENTITY(1,1)

[Documentacao oficial — CREATE TABLE](#)

[Documentacao oficial — Constraints \(PRIMARY KEY, FOREIGN KEY, CHECK\)](#)

[Documentacao oficial — ALTER TABLE](#)

DML — Operacoes CRUD

SELECT — Leitura de dados

```
USE uber_db;

-- Todas as corridas concluidas
SELECT * FROM corridas WHERE status = 'concluida';

-- TOP 10 corridas mais caras com alias e calculo
SELECT TOP 10
corrida_id,
valor_final,
distancia_km,
ROUND(valor_final / NULLIF(distancia_km,0), 2) AS preco_por_km
FROM corridas
WHERE status = 'concluida'
ORDER BY valor_final DESC;

-- Paginacao com OFFSET FETCH (pagina 2, 20 registros por pagina)
SELECT corrida_id, usuario_id, motorista_id, valor_final
FROM corridas
ORDER BY data_solicitacao DESC
OFFSET 20 ROWS FETCH NEXT 20 ROWS ONLY;
```

INSERT — Insercao eficiente

```
-- INSERT simples
INSERT INTO cidades (nome, estado, pais)
VALUES ('Sao Paulo', 'SP', 'BRA');

-- INSERT multiplo (uma unica operacao – mais eficiente)
INSERT INTO cidades (nome, estado)
VALUES
('Rio de Janeiro', 'RJ'),
('Curitiba', 'PR'),
('Belo Horizonte', 'MG'),
('Porto Alegre', 'RS');

-- INSERT com OUTPUT para capturar ID gerado
DECLARE @ids TABLE (id INT);
INSERT INTO cidades (nome, estado)
OUTPUT INSERTED.cidade_id INTO @ids
VALUES ('Recife', 'PE');
SELECT id FROM @ids;
```

UPDATE e DELETE com boas praticas

```
-- UPDATE seguro: sempre use WHERE especifico
UPDATE motoristas
SET ativo = 0
WHERE motorista_id = 7;

-- UPDATE com JOIN (recalcular avaliacao_media do motorista)
UPDATE m
SET m.avaliacao_media = sub.media
FROM motoristas m
JOIN (
  SELECT c.motorista_id,
  AVG(CAST(a.nota AS DECIMAL(3,2))) AS media
  FROM avaliacoes a
  JOIN corridas c ON a.corrida_id = c.corrida_id
  WHERE a.avaliador_tipo = 'usuario'
  GROUP BY c.motorista_id
) sub ON m.motorista_id = sub.motorista_id;

-- Boa pratica: SELECT antes do DELETE para revisar o que sera removido
SELECT * FROM promocoos
WHERE validade < CAST(GETDATE() AS DATE)
AND usos_realizados = 0;

DELETE FROM promocoos
WHERE validade < CAST(GETDATE() AS DATE)
AND usos_realizados = 0;
```

Atencao: Sempre execute um SELECT com o mesmo WHERE antes de um UPDATE ou DELETE em producao. Use BEGIN TRANSACTION + ROLLBACK para testar modificacoes sem confirmar irreversivelmente.

Operadores de filtragem — Referencia

Operador	Descricao	Exemplo
BETWEEN	Faixa de valores (inclusiva)	valor_final BETWEEN 20 AND 100
IN	Lista de valores aceitos	status IN ('concluida', 'cancelada')
LIKE	Padrao de texto	nome LIKE 'Jo%' -- inicia com Jo
IS NULL	Campo sem valor	data_fim IS NULL
EXISTS	Subquery retorna ao menos 1 linha	WHERE EXISTS (SELECT 1 FROM ...)
NOT IN	Exclui valores da lista	metodo NOT IN ('debito', 'pix')
AND / OR	Combinacao de condicoes	status='concluida' AND cidade_id=1

[Documentacao oficial — SELECT \(T-SQL\)](#)

[Documentacao oficial — UPDATE](#)

[Documentacao oficial — OFFSET FETCH \(Paginacao\)](#)

Consultas Avancadas

JOINS — Tipos e quando usar

Tipo de JOIN	Retorna	Caso de uso típico
INNER JOIN	Apenas linhas com correspondência em ambos os lados	Corridas com motorista ativo
LEFT JOIN	Todas as linhas da esquerda + correspondências (NULL se sem match)	Usuarios com ou sem corridas
RIGHT JOIN	Todas as linhas da direita + correspondências	Raramente usado; prefira LEFT JOIN
FULL OUTER JOIN	Todas as linhas de ambas as tabelas	Auditoria de dados orfaos
CROSS JOIN	Produto cartesiano (todas as combinações)	Tabelas de combinações / gerador de datas
SELF JOIN	Tabela juntada com ela mesma	Hierarquias, referências entre linhas da mesma tabela

Exemplo: detalhamento completo de uma corrida


```

SELECT
  c.corrida_id,
  u.nome AS passageiro,
  m.nome AS motorista,
  m.avaliacao_media,
  CONCAT(v.marca, ' ', v.modelo) AS veiculo,
  v.categoria,
  ci.nome AS cidade,
  c.origem_endereco,
  c.destino_endereco,
  c.distancia_km,
  c.valor_final,
  c.surge_multiplier,
  p.metodo AS forma_pagamento,
  p.status AS status_pagamento,
  DATEDIFF(MINUTE, c.data_inicio, c.data_fim) AS duracao_real_min
FROM corridas c
INNER JOIN usuarios u ON c.usuario_id = u.usuario_id
INNER JOIN motoristas m ON c.motorista_id = m.motorista_id
INNER JOIN veiculos v ON c.veiculo_id = v.veiculo_id
INNER JOIN cidades ci ON c.cidade_id = ci.cidade_id
LEFT JOIN pagamentos p ON c.corrida_id = p.corrida_id
WHERE c.status = 'concluida'
ORDER BY c.data_fim DESC;

```

Subqueries e CASE WHEN

```
-- Subquery no WHERE: motoristas com media abaixo de 4
SELECT motorista_id, nome, avaliacao_media
FROM motoristas
WHERE motorista_id IN (
  SELECT c.motorista_id
  FROM avaliacoes a
  JOIN corridas c ON a.corrida_id = c.corrida_id
  WHERE a.avaliador_tipo = 'usuario'
  GROUP BY c.motorista_id
  HAVING AVG(CAST(a.nota AS FLOAT)) < 4.0
);

-- CASE WHEN: classificar corridas por distancia
SELECT corrida_id, distancia_km,
CASE
  WHEN distancia_km < 5 THEN 'Curta'
  WHEN distancia_km < 20 THEN 'Media'
  ELSE 'Longa'
END AS classificacao
FROM corridas WHERE status = 'concluida';
```

Funcoes nativas mais utilizadas

Categoria	Funcao	Exemplo pratico
Data/Hora	DATEDIFF(unit, d1, d2)	DATEDIFF(MINUTE, data_inicio, data_fim)
Data/Hora	DATEADD(unit, n, date)	DATEADD(DAY, -30, GETDATE())
Data/Hora	FORMAT(date, pattern)	FORMAT(data_inicio, 'dd/MM/yyyy HH:mm')
Data/Hora	DATEPART(part, date)	DATEPART(HOUR, data_solicitacao)
Texto	LEN / DATALENGTH	LEN(comentario)
Texto	CONCAT(a, b, ...)	CONCAT(m.marca, ' ', m.modelo)
Texto	TRIM / UPPER / LOWER	TRIM(email)
Numerico	ROUND(n, decimals)	ROUND(valor_final, 2)
Numerico	NULLIF(a, b)	NULLIF(distancia_km, 0) — evita divisao por zero
Nulidade	ISNULL(col, default)	ISNULL(comentario, 'Sem comentario')
Nulidade	COALESCE(a, b, c)	COALESCE(data_fim, GETDATE())

[Documentacao oficial — JOINS e performance](#)

[Documentacao oficial — Funcoes de data e hora](#)

[Documentacao oficial — Subqueries](#)

Indices e Performance

Indices são estruturas de disco que permitem ao SQL Server localizar linhas sem varrer toda a tabela. Um índice errado pode piorar a performance de escritas; um índice bem projetado pode reduzir leituras de centenas de milhares de páginas para dezenas.

Tipos de índice

Tipo	Estrutura	Quando usar
Clustered Index	Define a ordem física dos dados (1 por tabela)	Coluna usada em range scans e ORDER BY — geralmente a PK
Non-Clustered Index	Estrutura separada com ponteiro para a linha	Colunas em WHERE, JOIN e ORDER BY de queries frequentes
Índice Composto	Múltiplas colunas no mesmo índice	Queries que filtram por 2+ colunas juntas
Índice com INCLUDE	Colunas extras na folha do índice	Evita Key Lookup — covering index
Índice Único	Garante unicidade + performance de busca	email, CPF, placa
Índice Filtrado	Índice parcial com cláusula WHERE	Indexar apenas registros ativos ou por status específico

Criando índices estratégicos para o dataset Uber

```
-- Índice para busca de corridas por usuário e data (query mais frequente)
CREATE NONCLUSTERED INDEX IX_corridas_usuario_data
ON corridas (usuario_id, data_solicitacao DESC)
INCLUDE (status, valor_final, motorista_id);

-- Índice para dashboard: corridas por cidade e status
CREATE NONCLUSTERED INDEX IX_corridas_cidade_status
ON corridas (cidade_id, status)
INCLUDE (valor_final, data_solicitacao);

-- Índice filtrado: apenas corridas ativas (aceita / em_andamento)
CREATE NONCLUSTERED INDEX IX_corridas_ativas
ON corridas (motorista_id, data_solicitacao)
WHERE status IN ('aceita', 'em_andamento');

-- Listar todos os índices de uma tabela
SELECT name, type_desc, is_unique, is_primary_key, filter_definition
FROM sys.indexes
WHERE object_id = OBJECT_ID('corridas');
```

Analizando performance com Execution Plan e STATISTICS IO

```
-- Ativar estatísticas de I/O e tempo
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

-- Execução SEM índice adequado (observe Table Scan ou Index Scan no plano)
SELECT * FROM corridas WHERE status = 'concluida' AND cidade_id = 1;

-- Criar índice e executar novamente
CREATE NONCLUSTERED INDEX IX_corridas_status_cidade
ON corridas (status, cidade_id);

SELECT * FROM corridas WHERE status = 'concluida' AND cidade_id = 1;

-- Compare os 'logical reads' antes e depois
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;

-- Identificar índices não utilizados (candidatos a remoção)
SELECT OBJECT_NAME(i.object_id) AS tabela,
i.name AS indice,
s.user_seeks, s.user_scans, s.user_lookups, s.user_updates
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats s
ON i.object_id = s.object_id AND i.index_id = s.index_id
WHERE OBJECT_NAME(i.object_id) = 'corridas'
ORDER BY ISNULL(s.user_seeks,0) DESC;
```

Dica: No Azure Data Studio ou SSMS, pressione Ctrl+M (Include Actual Execution Plan) antes de executar para visualizar graficamente o custo de cada operação. Procure por 'Table Scan' e 'Key Lookup' — ambos indicam oportunidades de otimização com índices.

[Documentação oficial — Guia de design de índices](#)

[Documentação oficial — sys.dm_db_index_usage_stats](#)

Stored Procedures, Funcoes e Transactions

Stored Procedure com TRY/CATCH e controle transacional

```
CREATE OR ALTER PROCEDURE sp_iniciar_corrida
@usuario_id INT, @motorista_id INT, @veiculo_id INT,
@cidade_id INT, @origem NVARCHAR(200), @destino NVARCHAR(200),
@nova_corrida_id INT OUTPUT
AS
BEGIN
SET NOCOUNT ON;
BEGIN TRY
BEGIN TRANSACTION;
-- Validar motorista ativo
IF NOT EXISTS (
SELECT 1 FROM motoristas
WHERE motorista_id = @motorista_id AND ativo = 1)
THROW 50001, 'Motorista inativo ou nao encontrado.', 1;
-- Validar veiculo associado
IF NOT EXISTS (
SELECT 1 FROM veiculos
WHERE veiculo_id = @veiculo_id
AND motorista_id = @motorista_id AND ativo = 1)
THROW 50002, 'Veiculo nao associado ao motorista.', 1;
-- Inserir corrida
INSERT INTO corridas (
usuario_id, motorista_id, veiculo_id, cidade_id,
status, origem_endereco, destino_endereco)
VALUES (
@usuario_id, @motorista_id, @veiculo_id, @cidade_id,
'aceita', @origem, @destino);
SET @nova_corrida_id = SCOPE_IDENTITY();
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
THROW; -- Re-lanca o erro para o chamador
END CATCH
END;
```

```
-- Executar a SP com parametro de saida
DECLARE @id INT;
EXEC sp_iniciar_corrida
@usuario_id = 1, @motorista_id = 3, @veiculo_id = 3,
@cidade_id = 1, @origem = 'Av. Paulista, 1000',
@destino = 'Aeroporto de Congonhas',
@nova_corrida_id = @id OUTPUT;
SELECT @id AS corrida_criada;
```

Funcoes escalares e de tabela (TVF)

```
-- Funcao escalar: calcular valor com surge
CREATE OR ALTER FUNCTION fn_valor_com_surge
(@valor_base DECIMAL(10,2), @surge DECIMAL(4,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
RETURN ROUND(@valor_base * @surge, 2);
END;

-- Uso em SELECT
SELECT corrida_id,
dbo.fn_valor_com_surge(valor_base, surge_multiplier) AS valor_cobrado
FROM corridas WHERE status = 'concluida';

-- Table-Valued Function: historico de corridas de um usuario
CREATE OR ALTER FUNCTION fn_historico_usuario (@usuario_id INT)
RETURNS TABLE AS RETURN (
SELECT c.corrida_id, c.data_solicitacao, c.status,
m.nome AS motorista, v.modelo, v.categoria,
c.valor_final, p.metodo AS pagamento
FROM corridas c
JOIN motoristas m ON c.motorista_id = m.motorista_id
JOIN veiculos v ON c.veiculo_id = v.veiculo_id
LEFT JOIN pagamentos p ON c.corrida_id = p.corrida_id
WHERE c.usuario_id = @usuario_id
);

-- TVF usada como tabela
SELECT * FROM dbo.fn_historico_usuario(5)
ORDER BY data_solicitacao DESC;
```

[Documentacao oficial — CREATE PROCEDURE](#)

[Documentacao oficial — TRY...CATCH e THROW](#)

[Documentacao oficial — CREATE FUNCTION](#)

[Documentacao oficial — Transacoes \(BEGIN / COMMIT / ROLLBACK\)](#)

Aggregations e Window Functions

GROUP BY, HAVING e funcoes de agregacao

```
-- Receita e ticket medio por cidade e mes
SELECT
ci.nome AS cidade,
YEAR(c.data_fim) AS ano,
MONTH(c.data_fim) AS mes,
COUNT(*) AS total_corridas,
SUM(c.valor_final) AS receita_total,
AVG(c.valor_final) AS ticket_medio,
MAX(c.valor_final) AS corrida_mais_cara,
COUNT(DISTINCT c.motorista_id) AS motoristas_ativos
FROM corridas c
JOIN cidades ci ON c.cidade_id = ci.cidade_id
WHERE c.status = 'concluida'
GROUP BY ci.nome, YEAR(c.data_fim), MONTH(c.data_fim)
HAVING SUM(c.valor_final) > 1000
ORDER BY ano DESC, mes DESC, receita_total DESC;

-- ROLLUP: subtotais por cidade + total geral
SELECT
ISNULL(ci.nome, '** TOTAL GERAL **') AS cidade,
COUNT(*) AS total_corridas,
SUM(c.valor_final) AS receita
FROM corridas c
JOIN cidades ci ON c.cidade_id = ci.cidade_id
WHERE c.status = 'concluida'
GROUP BY ROLLUP(ci.nome);
```

Window Functions — analisar sem perder detalhes de linha


```

-- RANK: motoristas por receita dentro de cada cidade
SELECT
ci.nome AS cidade,
m.nome AS motorista,
SUM(c.valor_final) AS receita,
RANK() OVER (PARTITION BY ci.nome
ORDER BY SUM(c.valor_final) DESC) AS rank_cidade,
ROW_NUMBER()OVER (PARTITION BY ci.nome
ORDER BY SUM(c.valor_final) DESC) AS row_num
FROM corridas c
JOIN motoristas m ON c.motorista_id = m.motorista_id
JOIN cidades ci ON c.cidade_id = ci.cidade_id
WHERE c.status = 'concluida'
GROUP BY ci.nome, m.nome, m.motorista_id;

-- LAG e LEAD: comparar corrida atual com anterior do mesmo usuario
SELECT
usuario_id, corrida_id, data_solicitacao, valor_final,
LAG(valor_final) OVER
(PARTITION BY usuario_id ORDER BY data_solicitacao) AS corrida_anterior,
LEAD(valor_final) OVER
(PARTITION BY usuario_id ORDER BY data_solicitacao) AS proxima_corrida
FROM corridas WHERE status = 'concluida';

-- Running total de receita por data
SELECT
CAST(data_fim AS DATE) AS data,
SUM(valor_final) AS receita_dia,
SUM(SUM(valor_final)) OVER (ORDER BY CAST(data_fim AS DATE))
AS receita_acumulada
FROM corridas WHERE status = 'concluida'
GROUP BY CAST(data_fim AS DATE)
ORDER BY data;

```

Window Function	Comportamento	Diferença chave
ROW_NUMBER()	Numero sequencial unico por janela	Nunca repete, mesmo com empates
RANK()	Mesmo numero para empates, pula o proximo	1, 1, 3 (pula o 2)
DENSE_RANK()	Mesmo numero para empates, nao pula	1, 1, 2 (nao pula)
LAG(col, n)	Valor da linha N posicoes antes	Acessa linha anterior na mesma janela
LEAD(col, n)	Valor da linha N posicoes adiante	Acessa proxima linha na mesma janela

SUM() OVER	Soma acumulada ou por particao	Nao colapsa linhas como GROUP BY
------------	--------------------------------	----------------------------------

[Documentacao oficial — Window Functions \(clausula OVER\)](#)

[Documentacao oficial — GROUP BY com ROLLUP, CUBE, GROUPING SETS](#)

Views, CTEs e Queries Avancadas

Views — encapsulando consultas complexas

```
CREATE OR ALTER VIEW vw_corridas_completas AS
SELECT
  c.corrida_id,
  u.nome AS passageiro,
  u.email AS email_passageiro,
  m.nome AS motorista,
  m.avaliacao_media,
  v.categoria,
  CONCAT(v.marca, ' ', v.modelo) AS veiculo,
  ci.nome AS cidade,
  c.status,
  c.origem_endereco, c.destino_endereco,
  c.distancia_km, c.valor_final,
  c.surge_multiplier,
  p.metodo AS pagamento,
  p.status AS status_pgto,
  DATEDIFF(MINUTE, c.data_inicio, c.data_fim) AS duracao_min
FROM corridas c
JOIN usuarios u ON c.usuario_id = u.usuario_id
JOIN motoristas m ON c.motorista_id = m.motorista_id
JOIN veiculos v ON c.veiculo_id = v.veiculo_id
JOIN cidades ci ON c.cidade_id = ci.cidade_id
LEFT JOIN pagamentos p ON c.corrida_id = p.corrida_id;

-- Usar a view como tabela
SELECT cidade, categoria, AVG(valor_final) AS ticket_medio
FROM vw_corridas_completas
WHERE status = 'concluida'
GROUP BY cidade, categoria;
```

CTEs — Common Table Expressions

```

-- CTE multipla: top 3 motoristas por cidade
WITH cte_receita AS (
  SELECT c.motorista_id, c.cidade_id,
  SUM(c.valor_final) AS receita_total,
  COUNT(*) AS total_corridas
  FROM corridas c WHERE c.status = 'concluida'
  GROUP BY c.motorista_id, c.cidade_id
),
cte_ranking AS (
  SELECT *,
  RANK() OVER (PARTITION BY cidade_id
  ORDER BY receita_total DESC) AS rank
  FROM cte_receita
)
SELECT m.nome, ci.nome AS cidade,
r.receita_total, r.total_corridas, r.rank
FROM cte_ranking r
JOIN motoristas m ON r.motorista_id = m.motorista_id
JOIN cidades ci ON r.cidade_id = ci.cidade_id
WHERE r.rank <= 3
ORDER BY ci.nome, r.rank;

```

MERGE e PIVOT

```

-- MERGE: atualizar avaliacao_media de todos os motoristas
MERGE motoristas AS alvo
USING (
SELECT c.motorista_id,
AVG(CAST(a.nota AS DECIMAL(3,2))) AS nova_media
FROM avaliacoes a
JOIN corridas c ON a.corrida_id = c.corrida_id
WHERE a.avalizador_tipo = 'usuario'
GROUP BY c.motorista_id
) AS origem ON (alvo.motorista_id = origem.motorista_id)
WHEN MATCHED THEN
UPDATE SET alvo.avaliacao_media = origem.nova_media;

-- PIVOT: corridas por cidade x categoria de veiculo
SELECT cidade_id, UberX, UberBlack, UberComfort, Moto
FROM (
SELECT c.cidade_id, v.categoria
FROM corridas c
JOIN veiculos v ON c.veiculo_id = v.veiculo_id
WHERE c.status = 'concluida'
) AS dados
PIVOT (
COUNT(categoria)
FOR categoria IN ([UberX],[UberBlack],[UberComfort],[Moto])
) AS pvt;

```

[Documentacao oficial — WITH \(CTE\)](#)

[Documentacao oficial — MERGE](#)

[Documentacao oficial — PIVOT e UNPIVOT](#)

Exportacao, Backup e Automacao

Exportacao com BCP e BULK INSERT

```
-- Exportar tabela corridas para CSV (terminal)
bcp uber_db.dbo.corridas out C:\exports\corridas.csv \
-c -t',' -r'\n' -S localhost -U sa -P UberWorkshop@2024

-- Exportar resultado de query customizada
bcp "SELECT c.corrida_id, u.nome, c.valor_final
FROM uber_db.dbo.corridas c
JOIN uber_db.dbo.usuarios u ON c.usuario_id = u.usuario_id
WHERE c.status = 'concluida'"
queryout C:\exports\relatorio.csv \
-c -t',' -S localhost -U sa -P UberWorkshop@2024

-- Importar CSV com BULK INSERT (T-SQL)
BULK INSERT uber_db.dbo.corridas
FROM 'C:\dados\corridas_importacao.csv'
WITH (
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    FIRSTROW = 2,
    TABLOCK
);
```

Backup e Restore — Estrategia 3-2-1

```
-- Backup FULL (base de toda a estrategia)
BACKUP DATABASE uber_db
TO DISK = 'C:\backups\uber_db_full.bak'
WITH FORMAT, INIT, COMPRESSION,
NAME = 'uber_db - Full Backup',
STATS = 10;

-- Backup DIFERENCIAL (somente mudancas desde o ultimo full)
BACKUP DATABASE uber_db
TO DISK = 'C:\backups\uber_db_diff.bak'
WITH DIFFERENTIAL, NAME = 'uber_db - Differential';

-- Backup de LOG DE TRANSACOES
BACKUP LOG uber_db
TO DISK = 'C:\backups\uber_db_log.trn';

-- Verificar integridade antes de restaurar
RESTORE VERIFYONLY FROM DISK = 'C:\backups\uber_db_full.bak';

-- Restaurar em novo banco (com remapeamento de arquivos)
RESTORE DATABASE uber_db_restore
FROM DISK = 'C:\backups\uber_db_full.bak'
WITH MOVE 'uber_db' TO 'C:\data\uber_restore.mdf',
MOVE 'uber_db_log' TO 'C:\data\uber_restore.ldf',
REPLACE, STATS = 5;
```

Tipo de Backup	O que captura	Frequencia tipica	RPO resultante
FULL	Todo o banco	Semanal	7 dias
DIFFERENTIAL	Mudancas desde o ultimo FULL	Diario	1 dia
LOG	Transacoes desde o ultimo LOG	A cada 15-60 min	Minutos

[Documentacao oficial — BACKUP DATABASE](#)

[Documentacao oficial — BCP Utility](#)

[Documentacao oficial — BULK INSERT](#)

SQL Server com Docker

docker-compose.yml completo com SQL Server + Adminer

```
# docker-compose.yml
version: '3.8'
services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: uber_sqlserver
    env_file: .env
    environment:
      ACCEPT_EULA: 'Y'
      MSSQL_PID: 'Developer'
    ports:
      - '1433:1433'
    volumes:
      - sqlserver_data:/var/opt/mssql
      - ./scripts:/scripts
    networks:
      - uber_net
    healthcheck:
      test: /opt/mssql-tools/bin/sqlcmd -S localhost -U SA
        -P $SA_PASSWORD -Q 'SELECT 1' || exit 1
      interval: 30s
      timeout: 10s
      retries: 5
  adminer:
    image: adminer:latest
    container_name: uber_adminer
    depends_on:
      sqlserver:
        condition: service_healthy
    ports:
      - '8080:8080'
    networks:
      - uber_net
    volumes:
      sqlserver_data:
    networks:
      uber_net:
    driver: bridge
```


Arquivo .env e inicializacao automatica

```
# .env (NUNCA commite este arquivo no git)
SA_PASSWORD=UberWorkshop@2024

# Subir o ambiente
docker-compose up -d

# Executar scripts SQL dentro do container
docker exec uber_sqlserver /opt/mssql-tools/bin/sqlcmd \
-S localhost -U SA -P UberWorkshop@2024 \
-i /scripts/01_create_schema.sql

docker exec uber_sqlserver /opt/mssql-tools/bin/sqlcmd \
-S localhost -U SA -P UberWorkshop@2024 \
-i /scripts/02_seed_data.sql

# String de conexao por linguagem:
# Python (pyodbc):
# 'DRIVER={ODBC Driver 18};SERVER=localhost,1433;DATABASE=uber_db;
# UID=SA;PWD=UberWorkshop@2024;TrustServerCertificate=yes'
# Node.js (mssql):
# { server:'localhost', port:1433, database:'uber_db',
# authentication:{type:'default', options:{userName:'SA',
# password:'UberWorkshop@2024'}} }
# .NET / C#:
# Server=localhost,1433;Database=uber_db;User Id=SA;
# Password=UberWorkshop@2024;TrustServerCertificate=True;
```

Dica: Organize seus scripts em uma pasta ./scripts/ numerada: 01_create_schema.sql, 02_seed_data.sql, 03_indexes.sql, 04_procedures.sql. Isso permite reconstruir o ambiente completo com um unico 'docker-compose up'.

[Documentacao oficial — SQL Server no Docker \(Linux\)](#)

[Imagem oficial — mcr.microsoft.com/mssql/server](https://mcr.microsoft.com/mssql/server)

Referencia Rapida — Documentacao Oficial Microsoft

[Portal principal da documentacao T-SQL](#)

[Tipos de dados](#)

[CREATE TABLE + Constraints](#)

[SELECT completo](#)

[JOINS e performance](#)

[Window Functions \(OVER\)](#)

[GROUP BY \(ROLLUP, CUBE, GROUPING SETS\)](#)

[WITH CTE](#)

[MERGE \(Upsert\)](#)

[PIVOT e UNPIVOT](#)

[CREATE PROCEDURE](#)

[CREATE FUNCTION](#)

[TRY...CATCH e THROW](#)

[Transacoes \(BEGIN/COMMIT/ROLLBACK\)](#)

[Guia de design de indices](#)

[BACKUP DATABASE](#)

[BCP Utility](#)

[BULK INSERT](#)

[SQL Server no Docker](#)

[Azure Data Studio](#)

[sys.dm_db_index_usage_stats](#)

[Funcoes de data e hora](#)